



StackMaster Tech

CLOUD · DATA · OBSERVABILITY · SRE

FIELD HANDBOOK · DAY 3 · SPLUNK POWER USER TRACK

Alerts, Macros & Acceleration

Operationalising search: turning saved searches into **alerts** that act, packaging SPL into reusable **macros**, and making it all fast with report acceleration, summary indexing, data models and **tstats**.

Alerts & Actions

Lookups in Alerts

Webhooks

Nested Macros

Report Accel · Summary Index

Data Models · tstats

Vivek Arora

Corporate Trainer, Author & Consultant
StackMaster Tech · Bengaluru, India

youtube.com/@StackMasterTech
github.com/vivek9325
linkedin.com/in/vivek11arora

Contents

ORIENTATION

From Ad-hoc Search to Operations	01
----------------------------------	----

PART A · EXPLORING ALERTS

1 · Referencing Lookups in Alerts	02
2 · Outputting Alert Results to a Lookup	03
3 · Logging & Indexing Searchable Alert Events	04
4 · Webhook Alert Actions	05

PART B · ADVANCED SEARCH MACROS

5 · Nested Search Macros	06
6 · Previewing Search Macros Before Executing	07
7 · Tags & Event Types in Search Macros	08

PART C · ACCELERATION — REPORTS & SUMMARY INDEXING

8 · Report Acceleration	09
9 · Summary Indexing	10

PART D · ACCELERATION — DATA MODELS & TSIDX

10 · Exploring Data Models (datamodel)	11
11 · Data Model Acceleration	12
12 · tsidx Files & the tstats Command	13

APPENDICES

A · Acceleration Decision Flowchart	14
B · Day-3 Quick-Reference Cheat Sheet	15

01 · From Ad-hoc Search to Operations

Days 1 and 2 were about *asking questions*. Day 3 is about *standing up answers that run themselves*. A search you care about should be saved once and then reused four ways: as an **alert** that watches and acts, as a **macro** that other searches embed, as an **accelerated report** or **summary index** that returns instantly, and as a **data model** that powers Pivot and `tstats`. This day connects all four.



One saved search, four operational lives — all on the Buttercup Games dataset.

Fig 1.1 — The four destinations of a saved search.

The dataset, unchanged

We continue on the Buttercup Games data from Days 1–2: `index=web` (`access_combined` and `checkout_json`) and `index=security` (`linux_secure`). The planted brute-force burst from `203.0.113.7` becomes our alerting example; daily purchases become our summary-indexing and acceleration example.

ROLES & CAPABILITIES

Scheduling alerts, writing to summary indexes, and accelerating data models are governed by capabilities (`schedule_search`, `indexes_edit`, `accelerate_datamodel`). If a save option is greyed out, it's a role/permission issue, not a syntax one.

PART A

Exploring Alerts

An alert is a saved search plus a **schedule**, a **trigger condition**, and one or more **actions**. The search finds something; the condition decides if it matters; the action does something about it — enrich from a lookup, write results to a lookup, log a searchable event, or fire a webhook.

02 · Referencing Lookups in Alerts

Alerts get sharper when the search itself consults a lookup — to add context (severity, owner) or to filter to only what matters (known-bad, or explicitly *not* known-good). Because the lookup runs inside the scheduled search, the alert always evaluates against the current table.

Alert anatomy

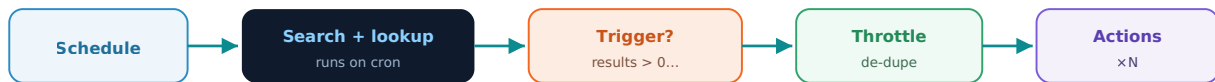


Fig 2.1 — Schedule → search (with lookup) → trigger condition → throttle → actions.

Example — only alert on failed logins from known-bad IPs

```
# the scheduled search behind the alert
index=security vendor_action="Failed password"
| lookup threat_ip_lookup src_ip OUTPUT threat_type, severity
| search threat_type=*
| stats count AS fails, values(severity) AS severity BY src_ip
| where fails >= 5
```

Set the alert to **trigger when number of results > 0**. It fires only when a source that appears in `threat_ip_lookup` also crosses the failure threshold — noise from unknown IPs never pages anyone.

Save As Alert — Brute-force SSH source

Alert type Scheduled ▼ Run on Cron Schedule ▼
Cron: */5 * * * * — every 5 minutes

Trigger condition Number of Results ▼ is greater than ▼ 0

Trigger Once per search · Throttle 60 min by src_ip

Trigger Actions

- **Add to Triggered Alerts** Severity High ▼
- **Output results to lookup** offender_watchlist.csv
- **Log Event** index summary source alert:brute_force
- **Webhook** https://hooks.slack.com/services/T00/B00/xxxx

Fig 2.2 — The Save-As-Alert dialog: schedule, trigger condition, throttle by `src_ip`, and four actions wired up.

THROTTLE TO STAY SANE

Without throttling, a 5-minute alert on an ongoing attack fires every 5 minutes. Suppress by `src_ip` for, say, 60 minutes so each offender pages you once per hour, not twelve times.

REAL-TIME VS SCHEDULED

Scheduled alerts run on a cron cadence and are cheap; real-time alerts watch a live stream continuously and cost a running search slot. Prefer scheduled unless you truly need sub-minute latency.

03 · Outputting Alert Results to a Lookup

An alert can *maintain state* by writing its results into a lookup. The classic pattern: an alert that detects offenders appends them to a watchlist, and other searches (and the alert itself) then reference that watchlist. The list is built by Splunk, from Splunk, automatically.

Two ways to write

Mechanism	Use
Trigger action "Output results to lookup"	No SPL — pick a lookup in the alert's action list; results overwrite it each fire.
<code>outputlookup</code> in the search	Full control: <code>append=true</code> , <code>key_field=</code> for upserts, or write a KV Store collection.

Example — build/refresh the offender watchlist

```
index=security vendor_action="Failed password"
| sort 0 _time | streamstats time_window=5m count AS w BY src_ip
| stats count AS fail_count, max(w) AS peak_5m BY src_ip | sort - fail_count
| outputlookup offender_watchlist.csv
```

index=security vendor_action="Failed password"			Last 24 hours ▼	
<pre> sort 0 _time streamstats time_window=5m count AS w BY src_ip stats count AS fail_count, max(w) AS peak_5m BY src_ip sort - fail_count outputlookup offender_watchlist.csv</pre>				
Events	Patterns	Statistics	Visualization	
written to offender_watchlist.csv				
src_ip %	fail_count %	peak_5m %		
203.0.113.7	14	14		
220.11.231.199	1	1		
148.120.226.163	1	1		
36.134.253.222	1	1		
171.184.47.24	1	1		
183.230.252.140	1	1		

Fig 3.1 — Results written to offender_watchlist.csv; 203.0.113.7 tops it. Other searches now `lookup` against this list.

OVERWRITE VS APPEND

A plain `outputlookup` (and the trigger action)

REPLACES

the table. To grow a list over time use `append=true`, and de-duplicate with `| inputlookup + stats + outputlookup` or a KV Store `key_field` upsert.

04 · Logging & Indexing Searchable Alert Events

Alerts are more useful when their firings become **data**. Writing each triggered result back into an index turns "the alert went off" into a searchable, dashboardable, correlatable event — the foundation of notable-event workflows and alert analytics ("which alerts fire most? which hosts recur?").

Three ways to persist a firing

Method	What it does
Trigger action "Log Event"	Writes a single event (your text/fields) to a chosen index — no SPL.
<code> collect index=summary</code>	Writes the search's result rows as events into a summary index, tagged with <code>source</code> / <code>sourcetype</code> .
Adaptive Response / Notable	In Enterprise Security, creates a notable event with owner, status and severity.

Example — record each brute-force firing as an event

```
index=security vendor_action="Failed password"
| sort 0 _time | streamstats time_window=5m count AS peak_5m BY src_ip
| where peak_5m >= 5
| eval alert_name="brute_force_ssh_source", severity="high"
| collect index=summary source="alert:brute_force"
```

Then treat firings like any other data

```
index=summary source="alert:brute_force"
| table _time alert_name src_ip severity peak_5m
```

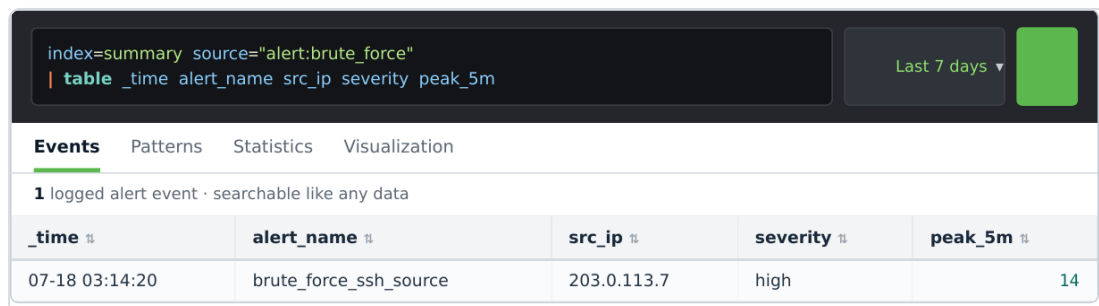


Fig 4.1 — The firing is now a searchable event; build trend dashboards and correlation on top of it.

WHERE FIRINGS ALREADY LIVE

Splunk also records scheduler and alert-action activity in `index=_internal` (`sourcetype=scheduler`) and the audit trail in `index=_audit`. Great for "did my alert run / fire?" troubleshooting even without a custom summary event.

05 · Webhook Alert Actions

A **webhook** action makes Splunk POST a JSON payload to an HTTP endpoint when the alert fires — the bridge to Slack, Microsoft Teams, PagerDuty, ServiceNow, or your own automation. No add-on required; it's built in.

What Splunk sends

On trigger, Splunk POSTs a JSON body describing the alert and its first result, e.g.:

```
{
  "sid": "scheduler_admin_search_brute_force...",
  "search_name": "brute_force_ssh_source",
  "result": { "src_ip": "203.0.113.7", "peak_5m": "14" },
  "results_link": "https://splunk:8000/app/search/@go?sid=..."
}
```

Configure it

Field	Value
Trigger action	Webhook
URL	<code>https://hooks.slack.com/services/T00/B00/xxxx</code>
Trigger	For each result (per row) or once per alert

The receiver reads `result.src_ip` and `result.peak_5m` and posts a formatted message with a deep link back to the search.

SECURITY & EGRESS

Webhooks call out from the search head — the target must be reachable and, for internet endpoints, allowed through egress/proxy. Prefer HTTPS, avoid putting secrets in the URL, and use per-result triggering deliberately (a noisy alert can flood the endpoint).

RICHER PAYLOADS

Need more than the default body? Use a custom alert action (a small app with an `alert_actions.conf` stanza and a script) or route through Splunk's HTTP Event Collector / an automation platform for full control of the message shape.

PART B

Advanced Search Macros

A macro is a named, reusable chunk of SPL — optionally parameterised — that expands in place before the search runs. Macros keep complex logic in one place, so a fix propagates everywhere. Here we nest them, preview their expansion, and combine them with tags and event types for portable, source-independent searches.

06 • Nested Search Macros

Macros are defined under **Settings** → **Advanced search** → **Search macros** (or `macros.conf`). You invoke one with backticks: ``name``, or ``name(arg)`` when it takes arguments. A macro's definition can itself call other macros — **nesting** — which is how you compose small, testable pieces into larger logic.

Define building blocks, then compose

Macro	Definition	Purpose
<code>revenue_by_day</code>	<code>bin _time span=1d stats sum(price) AS revenue BY _time</code>	Reusable daily rollup
<code>web_purchases(1)</code>	<code>index=web action=purchase categoryId=\$cat\$ `revenue_by_day`</code>	Filter + call the rollup (nested)

Use the composed macro

```
# one argument: the category
`web_purchases("SHOOTER")`
```

At runtime Splunk expands `web_purchases`, then expands the nested `revenue_by_day` inside it, producing the full pipeline — all from a two-token search.

ARGUMENTS & VALIDATION

Declare arguments in the macro (`$cat$`) and, for safety, add a

VALIDATION EXPRESSION

and error message so a bad argument fails clearly instead of producing a broken search. Arguments are string-substituted, so quote where the SPL needs quotes.

GUARD AGAINST LOOPS & INJECTION

Nesting is powerful but a macro must not (even indirectly) call itself — that's an infinite expansion. And because arguments are substituted verbatim, treat any user-supplied argument as untrusted: validate it, don't let it smuggle extra pipes.

07 · Previewing Search Macros Before Executing

Because a macro hides its body, you should **see the expansion** before you trust it — especially with nesting and arguments. Splunk expands macros in the search bar on demand so you can read the real SPL that will run.

Expand in place

Action	Result
Keyboard: Ctrl/Cmd + Shift + E	Opens the expansion preview for every macro in the search bar.
<code> noop</code> inspection	Run <code>... noop</code> and read the expanded search in the job's inspector.
Search job inspector	The <i>executed</i> search string shows all macros fully resolved.

Search macros — web_purchases(1)

Name

web_purchases(1)

one argument: cat

Definition

index=web action=purchase categoryId=\$cat\$ | `revenue\_by\_day`

Arguments

cat

Use & preview (⌘⇧E)

`web\_purchases("SHOOTER")`

expands to (nested macro revenue_by_day resolved too):

index=web action=purchase categoryId=SHOOTER | bin _time span=1d | stats sum(price) AS revenue BY _time

Fig 7.1 — The macro definition and its preview: ``web_purchases("SHOOTER")`` expands, nested `revenue_by_day` and all.

PREVIEW IS YOUR UNIT TEST

Always expand a parameterised or nested macro after editing it. The preview catches quoting mistakes, missing pipes, and wrong argument order *before* you run a heavy search or ship an alert built on it.

08 · Tags & Event Types in Search Macros

Event types name a search condition (`status>=400` → `web_error`); **tags** attach labels to field values or event types (`error` , `authentication`). Together they let you search by *meaning* instead of by raw predicates — and wrapping them in macros gives you a portable vocabulary that works across sources.

Define once

Object	Definition	Then search
eventtype web_error	index=web status>=400	eventtype=web_error
eventtype failed_ssh	index=security vendor_action="Failed password"	eventtype=failed_ssh
tag error	applied to web_error & server faults	tag=error

Wrap in a macro for a source-independent search

```
# macro: `problems` = (eventtype=web_error OR tag=error OR eventtype=failed_ssh)
`problems` | stats count BY eventtype
```

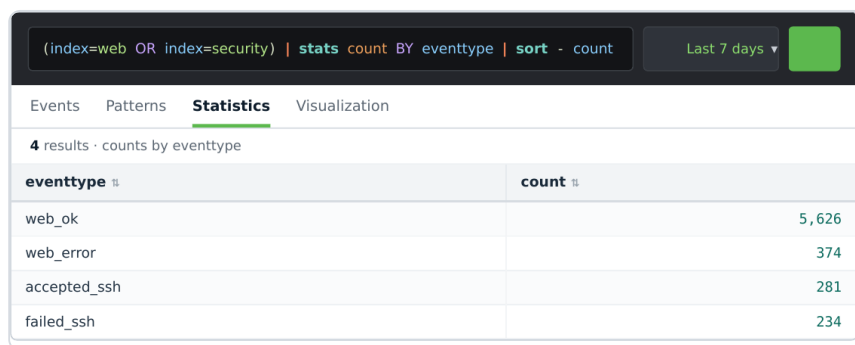


Fig 8.1 — Counting by eventtype across both indexes; the macro hides the raw predicates.

WHY THIS MATTERS FOR SCALE

When a new log source arrives, you point its events at the same event types and tags — every macro, alert and dashboard built on that vocabulary lights up for the new source with *zero* query changes. This is exactly the abstraction the CIM formalises.

COLOUR BY MEANING

Event types can carry a `color` and priority, so `web_error` events show consistently across every view — a small touch that makes dashboards instantly readable.

PART C

Acceleration — Reports & Summary Indexing

Big searches over long time ranges are slow because they re-read raw events every time. Acceleration pre-computes results so later searches read a small summary instead. Two classic techniques: **report acceleration** (Splunk manages the summary for you) and **summary indexing** (you roll up results into an index you control).

09 • Report Acceleration

Report acceleration is the low-effort option. You save a qualifying report, tick "Accelerate", pick a summary range (7/30/90 days...), and Splunk builds and maintains a background summary of intermediate results keyed to the search. Re-runs read the summary and finish in a fraction of the time — with no change to your SPL.

What qualifies

Requirement	Why
Uses a transforming command	<code>stats</code> / <code>timechart</code> / <code>chart</code> — the summary stores the aggregate, not raw events.
Transforming command is near the start	Only the pre-summary portion can be accelerated; keep heavy transforms early.
Runs in smart/fast mode	Verbose mode defeats the summary.
Owned/shared appropriately	Acceleration needs the right capability and app sharing.

Example — a report worth accelerating

```
# saved report: "Revenue by category by day" – accelerate over 90 days
index=web action=purchase
| timechart span=1d sum(price) AS revenue BY categoryId
```

After acceleration, opening this report over 90 days scans the summary, not 90 days of raw web events. Manage it under **Settings → Report acceleration summaries** (build status, completion %, size, and a rebuild button).

HOW IT STAYS FRESH

Splunk runs a background job every 10 minutes to fill in new time buckets. Summaries are stored alongside the index buckets; if you edit the report's SPL, the old summary is invalidated and rebuilt.

NOT A SILVER BULLET

Report acceleration ties a summary to *one* report's exact search. If you need the same pre-computation reused by *many* different searches, reach for summary indexing or a data model instead.

10 · Summary Indexing

Summary indexing is the flexible, portable option: a scheduled search computes a periodic rollup and writes the *results* as events into a dedicated `summary` index. Dashboards and searches then read the tiny summary index instead of raw data — and any number of them can share it.

The pattern

1. Create a `summary` index (Settings → Indexes).
2. Schedule a search that aggregates one period (e.g. hourly/daily) and writes results with `| collect` or the **si-** commands.
3. Search the summary index for fast long-range reporting.

Populate — daily revenue rollup

```
# scheduled daily, over "yesterday"; writes one summary event per category
index=web action=purchase earliest=-1d@d latest=@d
| stats sum(price) AS revenue, count AS orders BY categoryId
| collect index=summary report="daily_revenue"
```

Report — fast, long-range, from the summary

```
index=summary report="daily_revenue"
| timechart span=1d sum(revenue) AS revenue, sum(orders) AS orders
```

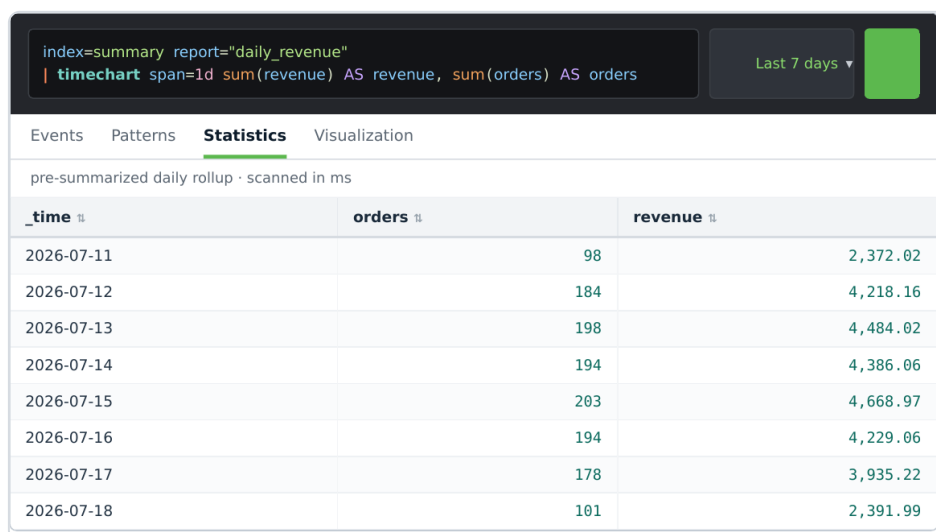


Fig 10.1 — Months of "revenue by day" read from a pre-summarised index in milliseconds.

AVOID GAPS & DOUBLE-COUNTS

Align the schedule and the time window exactly (a daily job over `-1d@d` to `@d`), and don't overlap windows or the same events get summed twice. Use the

SI-

commands (`sistats`, `sitimechart`) when you need statistically-correct partial aggregates that recombine safely.

REPORT ACCEL VS SUMMARY INDEX

Report acceleration is automatic but bound to one report; summary indexing is manual but reusable, portable, and you own the retention. Choose by how many consumers share the pre-computation.

PART D

Acceleration — Data Models & tsidx

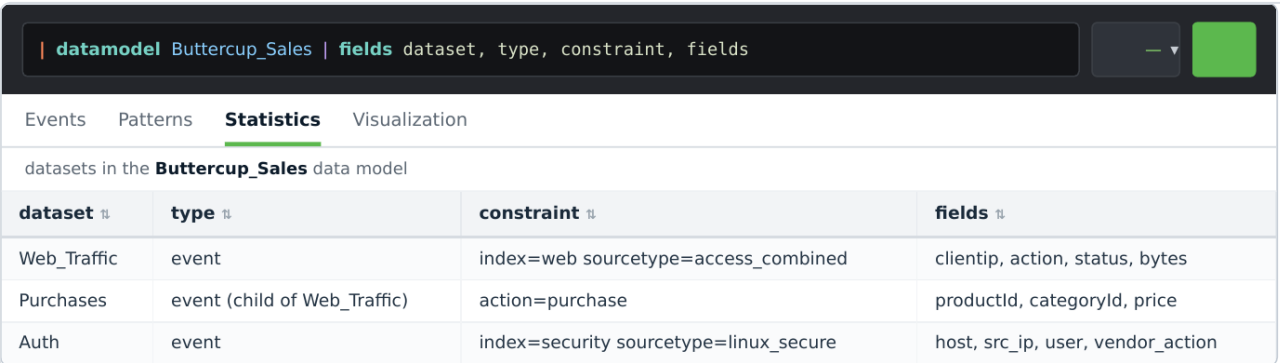
Data models turn raw events into a structured, hierarchical schema that powers Pivot and — when accelerated — extraordinarily fast `tstats` queries against compact **tsidx** summaries. This is how Enterprise Security searches billions of events in seconds.

11 · Exploring Data Models (datamodel)

A **data model** is a curated, object-oriented view of your data: named **datasets** (objects) arranged in a hierarchy, each with **constraints** (the search that defines membership) and **fields** (extracted, calculated, or looked-up). Analysts explore it in **Pivot** without writing SPL; power users query it with `datamodel` and `tstats`.

Inspect a model from the search bar

Command	Returns
datamodel	All data models available to you
datamodel Buttercup_Sales	The model's JSON: datasets, constraints, fields
datamodel Buttercup_Sales Purchases search	Runs the dataset as a search (raw events for that object)



dataset	type	constraint	fields
Web_Traffic	event	index=web sourcetype=access_combined	clientip, action, status, bytes
Purchases	event (child of Web_Traffic)	action=purchase	productId, categoryId, price
Auth	event	index=security sourcetype=linux_secure	host, src_ip, user, vendor_action

Fig 11.1 — Datasets of the Buttercup_Sales model: a Web_Traffic root, a Purchases child, and an Auth object.

Anatomy

- **Root dataset** — the base constraint (e.g. `index=web sourcetype=access_combined`).
- **Child dataset** — inherits the parent and adds a constraint (`action=purchase` → Purchases).
- **Fields** — auto (host, source), extracted (`productId`), or calculated (`revenue`).

MAP TO CIM

If you model toward the Common Information Model's standard datasets and field names, prebuilt Splunk apps and Enterprise Security correlation searches work against your data with no rework — the same payoff as tags/event types, at schema scale.

12 · Data Model Acceleration

Accelerating a data model tells Splunk to build and maintain **tsidx** summaries for the model's fields across a chosen time range. Once accelerated, `tstats` queries against the model read those columnar summaries instead of raw events — orders of magnitude faster.

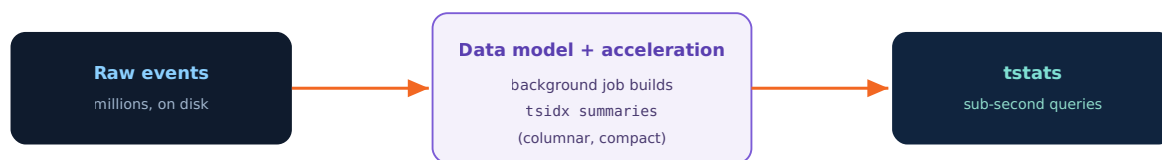


Fig 12.1 — Acceleration precomputes tsidx summaries that tstats reads directly.

Turn it on & check it

Where	What
Settings → Data models → Edit → Accelerate	Pick a summary range (e.g. 7 days); Splunk starts building.
Settings → Data models (expand row)	Build status, % complete, size, access count.
<code>tstats ... summariesonly=t</code>	Query only the accelerated summary (fast, but excludes unsummarised recent data unless <code>allow_old_summaries</code>).

COST & CONSTRAINTS

Acceleration consumes disk and background CPU, and the model must be in a stable, shared state (private models can't be accelerated). Accelerate the ranges you actually query — a 1-year summary you never search is pure overhead.

13 · tsidx Files & the tstats Command

Every Splunk bucket already has a **time-series index (tsidx)** — the structure that maps terms and field values to events. `tstats` runs statistics *directly* over these indexed structures (and over data-model acceleration summaries), skipping the expensive step of reading raw events. It's the fastest way to count and aggregate at scale.

tstats vs stats

Aspect	stats	tstats
Reads	Raw events (after search)	tsidx / DM acceleration summaries
Speed	Fast → slow with volume	Very fast, near-constant
Fields	Any search-time field	Indexed fields & DM fields
Syntax	after a base search	generating: starts with <code> </code> , uses <code>WHERE</code> / <code>BY</code>

Example — count by action, straight from the index

```
| tstats count WHERE index=web BY action
| sort - count
```

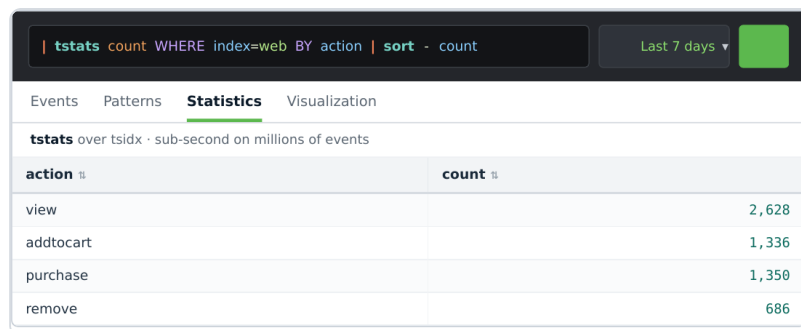


Fig 13.1 — The same numbers as a stats version, returned near-instantly from tsidx.

Against an accelerated data model

```
# time-bucketed revenue from the accelerated Buttercup_Sales model
| tstats sum(Purchases.price) AS revenue
  FROM datamodel=Buttercup_Sales.Purchases
  WHERE nodename=Purchases BY _time span=1d, Purchases.categoryId
```

PRESTATS FOR CHAINING

Add `prestats=t` to feed `tstats` output into `timechart` / `chart` for visualisations that stay fast: `| tstats prestats=t count BY _time span=1h | timechart count`.

WHEN TSTATS CAN'T HELP

`tstats` works on *indexed* fields and accelerated data-model fields. For arbitrary search-time fields on un-accelerated data, you still need `stats` — or accelerate the model first.

14 • Appendix A — Acceleration Decision Flowchart

Which acceleration fits the job? Follow how the pre-computation will be reused.

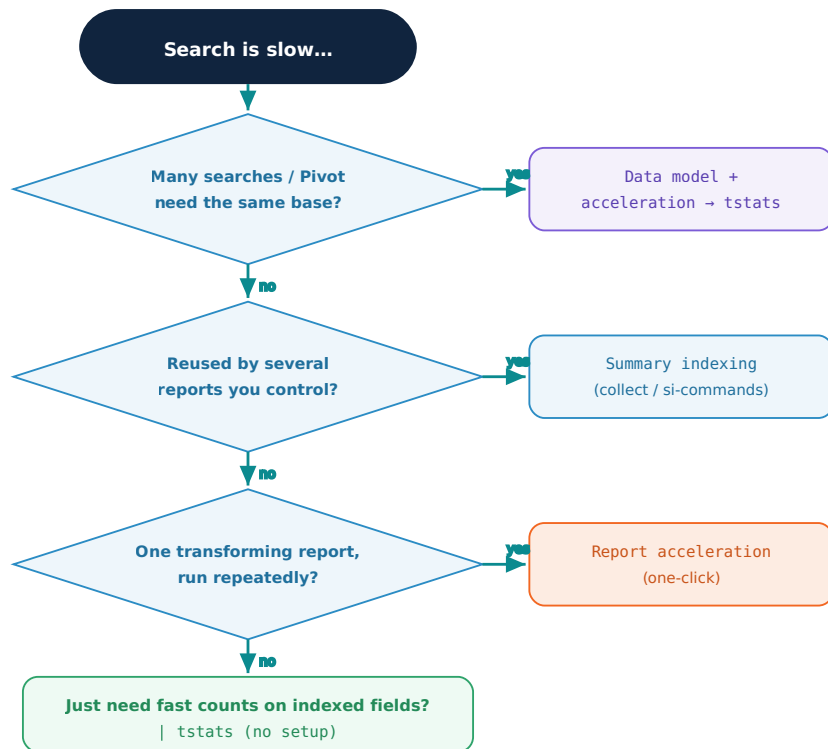


Fig A.1 — Match the acceleration to how the pre-computation is shared.

15 · Appendix B — Day-3 Quick-Reference Cheat Sheet

ALERTS

```
# lookup inside the alert search
| lookup threat_ip_lookup src_ip OUTPUT severity
| search severity=*
# write results to a lookup / watchlist
| outputlookup offender_watchlist.csv append=true
# log firing as a searchable event
| collect index=summary source="alert:brute_force"
```

ACTIONS

email · output-to-lookup · log event · webhook (POST JSON) · Add to Triggered Alerts. Throttle by a field to de-dupe.

MACROS

```
# invoke / with arg / nested
`revenue_by_day`
`web_purchases("SHOOTER")`
# preview expansion: Ctrl/Cmd + Shift + E
```

TAGS & EVENTTYPES

```
eventtype=web_error tag=error
eventtype=failed_ssh
```

ACCELERATION

```
# summary indexing: populate
... | stats ... | collect index=summary report="daily_revenue"
# statistically-safe partials
... | sstats sum(price) BY categoryId
# report acceleration → Save report → Accelerate
```

DATA MODELS & TSTATS

```
| datamodel Buttercup_Sales
| tstats count WHERE index=web BY action
| tstats sum(Purchases.price) FROM datamodel=Buttercup_Sales.Purchases BY _time span=1d
# chain to timechart
| tstats prestats=t count BY _time span=1h | timechart count
```

PICK YOUR ACCELERATION

Situation	Use
One report, rerun	Report accel
Shared rollup you own	Summary index
Pivot / many searches	DM acceleration
Fast counts, no setup	tstats

END OF DAY 3

You can now turn a search into an alert that enriches, writes lookups, logs searchable events and fires webhooks; package SPL into nested, previewable macros over tags and event types; and make everything fast with report acceleration, summary indexing, data models and tstats — all on the Buttercup Games dataset.